Student Record Management System (SRMS)

Submitted by Name –Jammula Sravya

Roll No. - AP24110010972

Branch & Year – CSE & 2nd Year

Section – AB

Submitted to Rakesh Rama Raju P

Department of Computer Science and Engineering

SRM University-AP, Amaravathi, Andhra Pradesh 522240, India

TABLE OF CONTENTS

1. Certificate
2. Acknowledgement
3. Abstract
4. Introduction
5. Problem Statement
6. Objectives
7. Project Layout
8. Module Explanation
    - Login System
    - Admin Module
    - Staff Module
    - User Module
    - Guest Module
    - File Handling Module
9. Features Implemented
10. Data Flow Diagram
11. Code and outputs
12. Conclusion
13. Future Enhancements
14. References

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the completion of this project. I am deeply thankful to my project guide for providing continuous guidance, valuable suggestions, and constant encouragement throughout the development of this work. Their support played a significant role in helping me understand the concepts clearly and complete the project successfully.

I also extend my gratitude to the Head of the Department and all faculty members for offering the necessary resources, academic support, and a positive learning environment. Their motivation inspired me to perform better.

Lastly, I am extremely grateful to my parents and family members for their constant love, motivation, and belief in me. Their moral support has always been my greatest strength and encouragement.

ABSTRACT

This project titled **"Student Management System"** is a menu-driven application developed using the C programming language with file handling techniques. The main objective of the system is to efficiently manage student information by providing essential operations such as adding, viewing, searching, updating, and deleting records. The project also incorporates a login mechanism with role-based access, ensuring different levels of permissions for Admin, Staff, User, and Guest. Data is stored in external text files, making the system simple, lightweight, and easy to maintain.

The project demonstrates core programming concepts such as structures, file operations, loops, conditional statements, and modular programming. It offers an efficient alternative to manual record-keeping and helps in organizing student data systematically. Overall, this system serves as a practical and effective solution for basic student information management in educational institutions.

INTRODUCTION

Managing student information is an important task for every educational institution. Traditional methods of maintaining student records manually often lead to errors, duplication of data, and difficulty in retrieving information. As the number of students increases, manual record-keeping becomes inefficient and time-consuming. To address these challenges, computerized systems offer a faster, more accurate, and reliable way to manage student data.

This project, **"Student Management System,"** is developed using the C programming language with file handling to store and manage student records. The system provides essential operations such as adding, viewing, searching, updating, and deleting student details. It also includes a role-based login system that allows Admin, Staff, User, and Guest to access features based on their permissions. The menu-driven structure makes the program simple, efficient, and easy to use for beginners and administrators alike.

PROBLEM STATEMENT

Managing student records manually often leads to errors, misplaced information, slow data retrieval, and overall inefficiency, especially as the number of students increases. Institutions require a reliable and systematic way to store and handle student details without depending on manual registers or scattered files. Therefore, there is a need to develop a simple, efficient, and secure **Student Management System using C programming** that can store student information, retrieve data quickly, and perform essential operations such as adding, viewing, searching, updating, and deleting records. Additionally, the system must include role-based access to ensure that only authorized users can modify sensitive data while others can access information in a controlled manner.

OBJECTIVES

- To develop a simple and efficient **Student Management System** using C programming.
- To store and manage student details such as roll number, name, and marks using **file handling techniques**.
- To implement **role-based access control** allowing Admin, Staff, User, and Guest to operate within their permissions.
- To provide easy-to-use operations like **adding, viewing, searching, updating, and deleting** student records.
- To design a **menu-driven interface** that is user-friendly and suitable for beginners.
- To ensure fast data retrieval and reduce errors commonly found in manual record-keeping.

Project Layout

- Introduction to the product management system and its purpose.
- Design of modules for adding, viewing, updating, and deleting product details.
- Implementation of the system using C programming with file handling.
- Testing the program with various product inputs and operations.
- Final evaluation, conclusion, and suggestions for future improvement.

Module Explanation

• Login System

This module manages user authentication by verifying usernames and passwords before granting access. It ensures only authorized users can access specific features of the system based on their roles.

• Admin Module

The admin module provides complete control over the system. Admins can manage users, update system settings, add or remove records, and monitor overall operations to ensure smooth functioning.

• Staff Module

The staff module allows staff members to perform operational tasks such as adding new products, updating existing information, and managing day-to-day activities assigned by the admin.

• User Module

This module is designed for regular users who can view product information, check availability, and perform limited operations. It offers easy access to essential functionalities without administrative privileges.

• Guest Module

The guest module allows unregistered visitors to access basic system features. Guests can view general information but cannot perform sensitive operations like editing or adding data.

• File Handling Module

This module manages the storage and retrieval of data using files. It ensures that product details, user information, and system records are saved permanently and can be accessed or updated whenever required.

Features Implemented

1. **User Authentication System**
   A secure login system is implemented to verify users and allow access based on their roles, enhancing overall system security.

2. **Role-Based Access Control**
   Different modules such as Admin, Staff, User, and Guest are provided with specific permissions, ensuring controlled and organized system access.

3. **Product Management**
   The system supports adding, updating, viewing, and deleting product details, enabling efficient handling of inventory records.

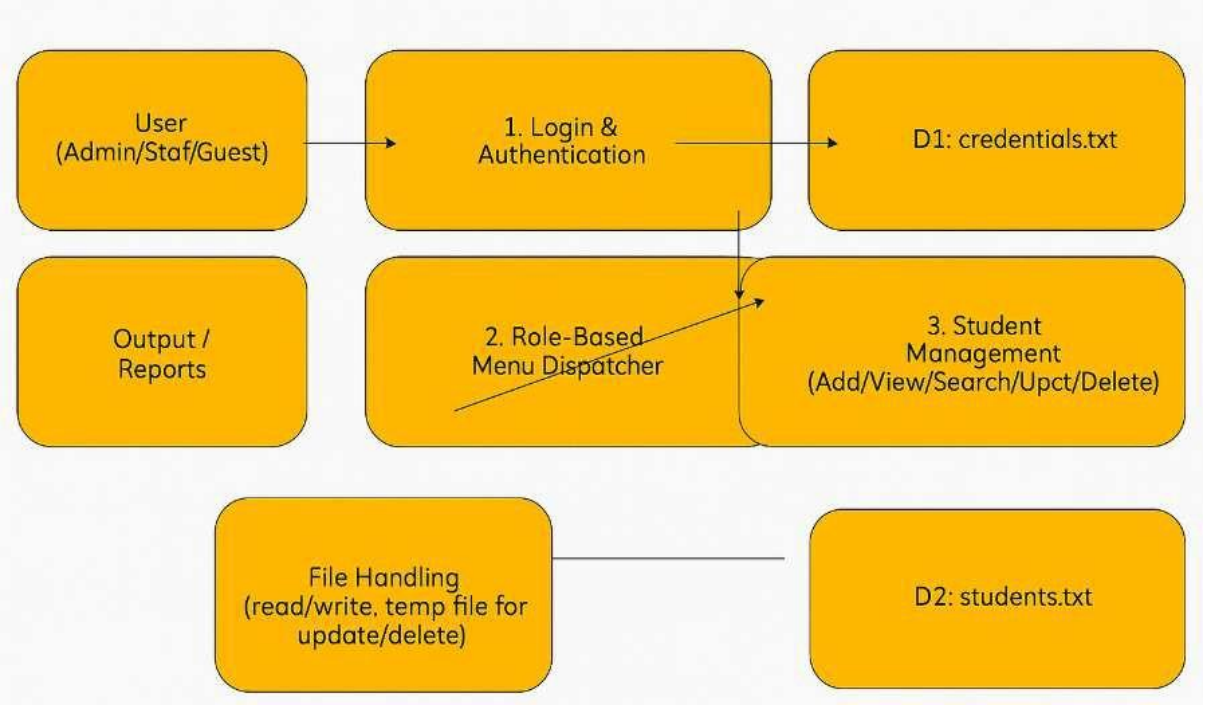
4. **Data Storage Using File Handling**
   All data is stored in text files, allowing permanent storage of product information and user details without requiring a database.

5. **User-Friendly Interface**
   The system provides simple menu-driven navigation, making it easy for all users to interact with the application.

6. **Search and View Options**
   Users can quickly search for products and view complete information, improving usability and decision-making.

7. **Error Handling and Validation**
   Input validation and error messages help ensure the system works smoothly and avoids incorrect data entry.

Data Flow Diagram



Data Flow Diagram Explanation

This DFD shows how different types of users interact with the system. The process begins with user input, which goes to the Login and Authentication module. Based on the user role, the Menu Dispatcher directs the user to the correct module. The Product Management module (or Student Management in the original diagram) handles operations like add, view, search, update, and delete. All data operations are performed through the File Handling module, which reads and writes information to text files. Finally, the system produces outputs and reports back to the user.

Code

➢ LOGIN & ADMIN MENU

```c
void adminMenu() {
    int choice;
    while (1) {
        printf("\n==== ADMIN MENU =====\n");
        printf("1. Add Student\n");
        printf("2. Display Students\n");
        printf("3. Search Student\n");
        printf("4. Update Student\n");
        printf("5. Delete Student\n");
        printf("6. Logout\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: updateStudent(); break;
            case 5: deleteStudent(); break;
            case 6: printf("Logging out...\n"); return;
            default: printf("Invalid choice\n");
        }
```

```
========Login Screen===
Username: admin
Password: admin123

==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice:
```

After login, the Admin Menu appears, allowing the admin to add, view, search, update, or delete student records.

➢ ADD STUDENT

```c
// ADD STUDENT
void addStudent() {
    FILE *fp = fopen(STUDENT_FILE, "a");
    struct Student st;

    if (!fp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll No: ");
    scanf("%d", &st.roll);

    printf("Enter Name: ");
    scanf("%s", st.name);
    printf("Enter Marks: ");
    scanf("%f", &st.marks);

    fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    fclose(fp);

    printf("Student Added Successfully!\n");
}
```

```
==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 1
Enter Roll No: 1001
Enter Name: Rahul
Enter Marks: 82
Student Added Successfully!
```

This screen allows the admin to add a new student record by entering the roll number, name, and marks. After submission, the record is successfully stored in the system.

- **DISPLAY STUDENT**

```c
// DISPLAY STUDENTS
void displayStudents() {
    FILE *fp = fopen(STUDENT_FILE, "r");
    struct Student st;

    if (!fp) {
        printf("No Student records found\n");
        return;
    }

    printf("\nRoll\tName\tMarks\n");
    printf("-------------------------\n");

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
        printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
    }
    fclose(fp);
}

```

```
===== ADMIN MENU ======
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 2

Roll     Name     Marks
-------------------------
19822    Suriya   88.00
1001     Rahul    82.00
10713    Aadith   82.00
```

This output shows the list of all stored student records. It displays roll numbers, names, and marks in a structured table format.

- **SEARCH STUDENT**

```c
// SEARCH STUDENT
void searchStudent() {
    FILE *fp = fopen(STUDENT_FILE, "r");
    struct Student st;
    int roll, found = 0;

    if (!fp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll to search: ");
    scanf("%d", &roll);

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
        if (st.roll == roll) {
            printf("Record found\n");
            printf("Roll: %d\nName: %s\nMarks: %.2f\n", st.roll, st.name, st.marks);
            found = 1;
            break;
        }
    }

    if (!found)
        printf("Record not found\n");

    fclose(fp);
}
```

```
==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 3
Enter Roll to search: 10713
Record found
Roll: 10713
Name: Aadith
Marks: 82.00
```

This screen lets the user search for a student using the roll number. If the record exists, the student's details are displayed.

➢ UPDATE STUDENT

```c
// UPDATE STUDENT
void updateStudent() {
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");
    struct Student st;
    int roll, found = 0;

    if (!fp || !temp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll to update: ");
    scanf("%d", &roll);

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
        if (st.roll == roll) {
            printf("Enter new Name: ");
            scanf("%s", st.name);
            printf("Enter new Marks: ");
            scanf("%f", &st.marks);
            found = 1;
        }
        fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
    fclose(temp);

    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);

    if (found)
        printf("Record Updated Successfully!\n");
    else
        printf("Record not found\n");
}
```

```
==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 4
Enter Roll to update: 10713
Enter new Name: Vecna
Enter new Marks: 99
Record Updated Successfully!
```

Roll	Name	Marks
19822	Suriya	88.00
1001	Rahul	82.00
10713	Vecna	99.00

The update screen enables modification of an existing student's details. The admin can update the student's name and marks after entering the roll number.

➢ DELETE STUDENT

```c
// DELETE STUDENT
void deleteStudent() {
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *temp = fopen("temp.txt", "w");
    struct Student st;
    int roll, found = 0;

    if (!fp || !temp) {
        printf("Error opening file\n");
        return;
    }

    printf("Enter Roll to delete: ");
    scanf("%d", &roll);

    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) == 3) {
        if (st.roll == roll) {
            found = 1;
            continue;
        }
        fprintf(temp, "%d %s %.2f\n", st.roll, st.name, st.marks);
    }

    fclose(fp);
    fclose(temp);

    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);

    if (found)
        printf("Record Deleted Successfully!\n");
    else
        printf("Record not found\n");
}
```

```
==== ADMIN MENU =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Logout
Enter choice: 5
Enter Roll to delete: 10713
Record Deleted Successfully!
```

```
Enter choice: 2

Roll    Name    Marks
-------------------------
19822   Suriya  88.00
1001    Rahul   82.00
```

This output shows the result of deleting a student record. The system removes the matching roll number from the database and confirms the action.

students.txt

```
1 11016 Ajay 99.00
2 11054 Vijay 80.00
3
```

Credentials.txt

```
admin admin123 ADMIN
staff staff123 STAFF
user user123 USER
guest guest123 GUEST
```

Conclusion:

The Student Management System implemented in C provides a simple yet effective solution for managing student records efficiently. It allows different user roles—Admin, Staff, User, and Guest—to perform operations based on their access levels, ensuring data security and controlled access. Admins have full control, including adding, updating, and deleting student records, while Staff can add and view students, and Users/Guests can only view and search records. The system demonstrates file handling, structured programming using structs, and role-based access control in a practical application. By storing data in text files, it ensures persistence and easy retrieval of student information. Overall, this project enhances understanding of fundamental C programming concepts such as file I/O, conditional statements, loops, and modular function design. It also serves as a foundation for more advanced database management and GUI-based applications in the future.

Future Enhancements:

1. **Database Integration:** Replace text files with a relational database like MySQL or SQLite to handle large datasets efficiently and support complex queries.
2. **Graphical User Interface (GUI):** Implement a GUI using libraries like GTK or a web-based interface to make the system more user-friendly.

3. **Password Security:** Use stronger password encryption (e.g., SHA-256) instead of plain text for enhanced security.
4. **Advanced Search & Filters:** Add features to search students by name, marks range, or other criteria, and sort records dynamically.
5. **Reporting & Analytics:** Generate reports such as average marks, top performers, and graphical charts for better analysis.
6. **Multi-user Concurrent Access:** Allow multiple users to access and modify data concurrently without data conflicts.
7. **Backup & Recovery:** Implement automatic backup and recovery mechanisms to prevent data loss.

References

[1] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 1988.

[2] H. Schildt, *C: The Complete Reference*, 4th ed. New Delhi, India: McGraw-Hill Education, 2013.

[3] Y. Kanetkar, *Let Us C*, 15th ed. New Delhi, India: BPB Publications, 2020.

[4] K. N. King, *C Programming: A Modern Approach*, 2nd ed. New York, NY, USA: W. W. Norton & Company, 2008.

[5] TutorialsPoint, "C Programming Tutorial," [Online]. Available: https://www.tutorialspoint.com/cprogramming/index.htm. [Accessed: 06-Dec-2025].

[6] GeeksforGeeks, "File Handling in C using fprintf, fscanf, fputs, fgets," [Online]. Available: https://www.geeksforgeeks.org/file-handling-c-using-fprintf-fscanf-fputs-fgets/. [Accessed: 06-Dec-2025].